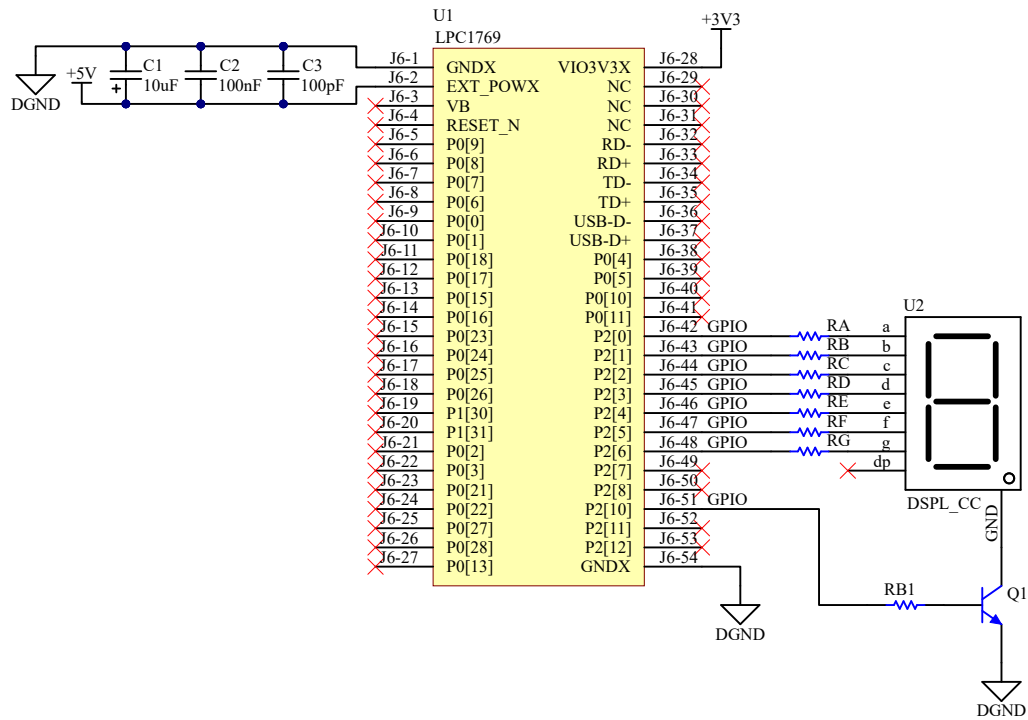


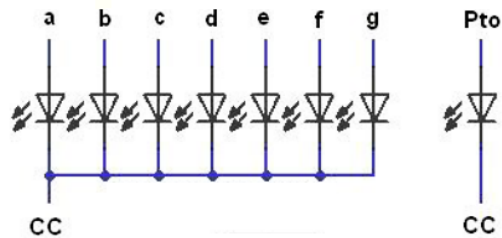
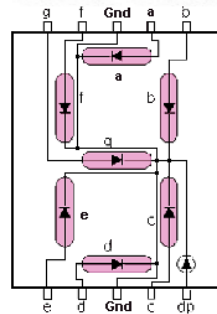
```

1  /*=====
2  * @file    Ejercicio_2_3.c
3  *
4  * @brief    Muestra de forma secuencial los dígitos del 0 al 9 en un display
5  *           de 7 segmentos conectado a los pines P2.0 a P2.7.
6  *
7  * @details  - Utilizar una tabla de consulta (LUT) para codificar los dígitos
8  *           del 0 al 9 en un display de 7 segmentos de cátodo común.
9  *           - Implementar retardo bloqueante.
10 *
11 * @author   Ing. E. Migliore
12 *
13 * @version  3.0
14  *=====*/

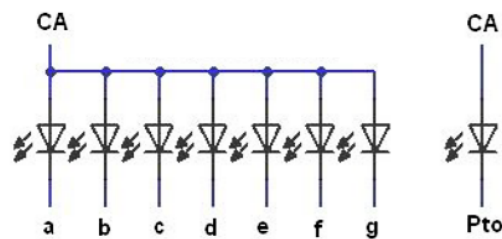
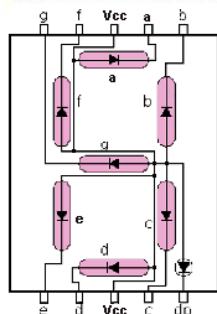
```



Cátodo Común



Anodo Común



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES
8  *=====*/
9  #define DSPL_MAX_DIGITS 10
10 #define DELAY_TIME 10000000
11 #define DATA_MASK_DSPL 0xFF
12 #define CTRL_DSPL (1 << 10)
13
14 /*=====
15 * DEFINICIÓN DE VARIABLES GLOBALES
16 *=====*/
17 const uint32_t lutDSPL_CC[10] =
18 {
19     0x3F, // 0
20     0x06, // 1
21     0x5B, // 2
22     0x4F, // 3
23     0x66, // 4
24     0x6D, // 5
25     0x7D, // 6
26     0x07, // 7
27     0x7F, // 8
28     0x67, // 9
29 };
30
31
32 /*=====
33 * PROTOTIPOS DE FUNCIONES
34 *=====*/
35 void cfgGPIO(void);
36 void countDigitDSPL(void);
37 void printDigitDSPL(uint8_t);
38 void delay(uint32_t);
39
40 /*=====
41 * FUNCIÓN PRINCIPAL
42 *=====*/
43 int main(void)
44 {
45     /*-----
46     * INICIALIZACIÓN DEL MCU
47     *-----*/
48     cfgGPIO();
49
50     /*-----
51     * INICIO PROGRAMA PRINCIPAL
52     *-----*/
53     while(1)
54     {
55         countDigitDSPL();
56     }
57
58     return 0;
59 }
60
61
62
63
64
65
66
67
68
69

```

```

70
71 /*=====
72  * DEFINICIONES DE FUNCIONES
73  *=====*/
74
75 /*****
76  * @brief   Configura los pines GPIO utilizados por el sistema.
77  *
78  * @details Configura los pines P2.0 a P2.7 como salidas digitales para
79  *          controlar los segmentos del display de 7 segmentos y P2.10
80  *          como salida digital para el control de una señal adicional.
81  *****/
82 void cfgGPIO(void)
83 {
84     LPC_GPIO2 -> FIODIR   |= (DATA_MASK_DSPL | CTRL_DSPL); /*P2.0-P2.7 y P2.10:
85                                                                Salidas digitales*/
86     LPC_GPIO2 -> FIOSET   |= CTRL_DSPL;                      // P2.10: Q1 ON
87
88     return;
89 }
90
91 /*****
92  * @brief   Ejecuta la cuenta de 0 a 9 en el display.
93  *
94  * @details Recorre la tabla de consulta digitDSPL y muestra secuencialmente
95  *          cada uno de los dígitos en el display de 7 segmentos.
96  *****/
97 void countDigitDSPL(void)
98 {
99     uint8_t i;
100
101     for(i = 0; i < DSPL_MAX_DIGITS; i++)
102     {
103         printDigitDSPL(lutDSPL[i]);
104         delay(DELAY_TIME);
105     }
106
107     return;
108 }
109
110 /*****
111  * @brief   Muestra un dígito en el display de 7 segmentos.
112  *
113  * @details Apaga previamente todos los segmentos y posteriormente establece
114  *          el patrón correspondiente al dígito recibido.
115  *
116  * @param   number Código de segmentos correspondiente al dígito a mostrar.
117  *****/
118 void printDigitDSPL(uint8_t digit)
119 {
120     LPC_GPIO2->FIOCLR = DATA_MASK_DSPL;    //P2.0-P2.7: Segmentos OFF
121     LPC_GPIO2->FIOSET = digit;               //P2.0-P2.7: Código del dígito
122
123     return;
124 }
125
126
127
128
129 /*****
130  * @brief   Genera un retardo bloqueante mediante instrucciones NOP.
131  *
132  * @details Ejecuta un lazo de espera durante la cantidad de iteraciones
133  *          indicada por delayTime. Cada iteración ejecuta una instrucción
134  *          NOP y la duración total depende del número de iteraciones y
135  *          de la frecuencia de CPU.
136  *
137  * @param   delayTime Cantidad de iteraciones del retardo.
138  *****/

```

```
139 void delay(uint32_t delayTime)
140 {
141     for(volatile uint32_t i = 0; i < delayTime; i++)
142     {
143         __NOP();    // NOP: 1 ciclo de CPU
144     }
145
146     return;
147 }
```